

AI 시스템 개요

시간표

	1일차	2일차
1		공정성
2	AI 시스템 개요	수행 효율성
3	AI 시스템 품질 개요	기능 적응성
4	기능 정확성	
5		
6	강건성	설명 용이성
7		AI 품질 우선 순위 및 Trade off
8	프라이버시	데이터 품질

목차

❖ AI 시스템

❖ AI Task

❖ AI Model

❖ Model/Software 배포

AI System: Functional View

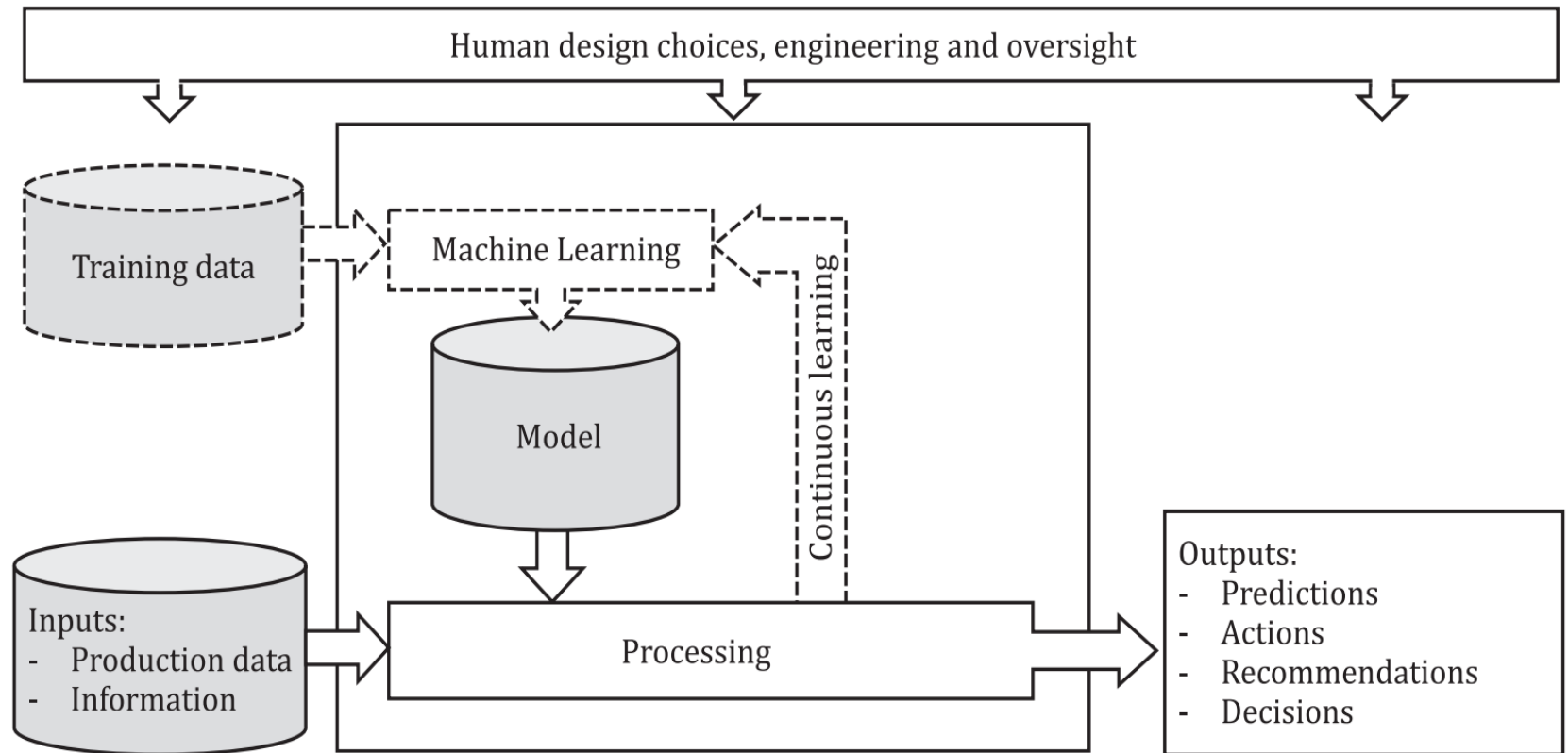
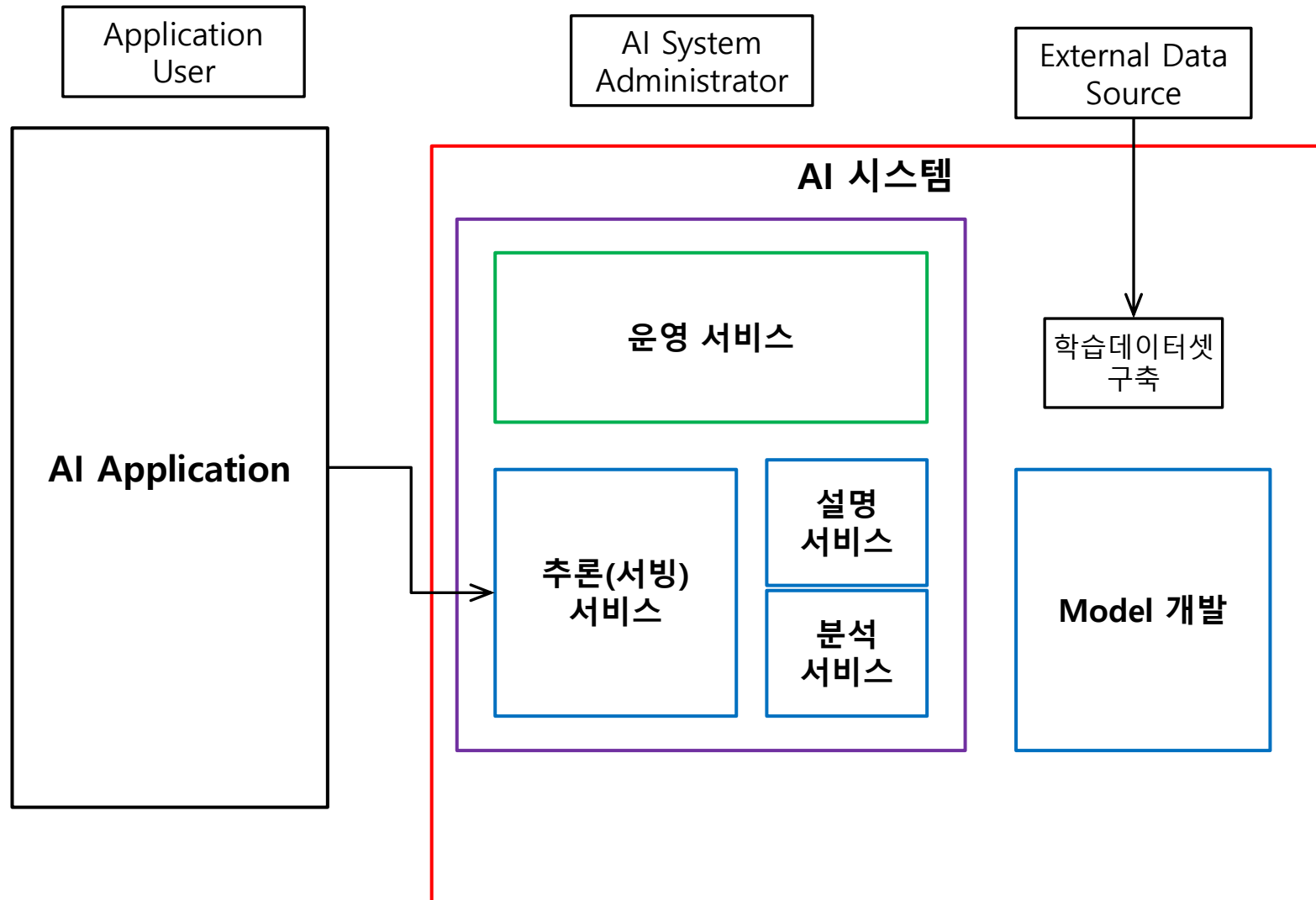
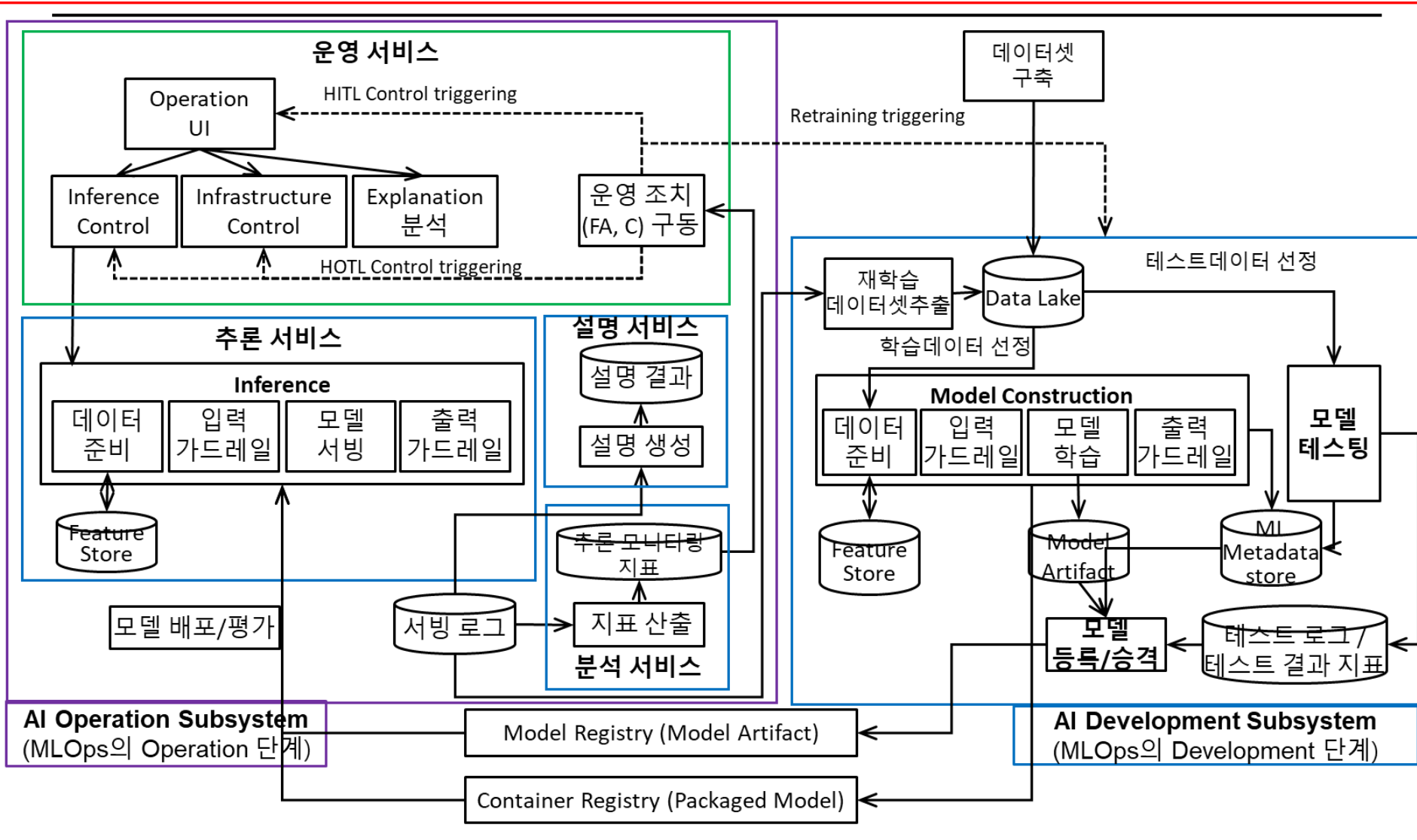


Figure 5 — AI system functional view

AI Application and AI System



AI System 아키텍처



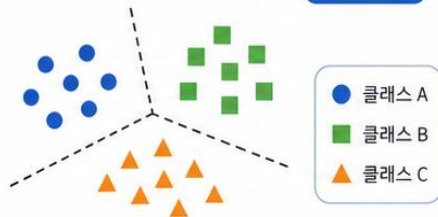
AI Task 유형

대표적인 AI Task 5가지

지도학습, 비지도학습, 패턴 탐지 및 개인화의 핵심 유형

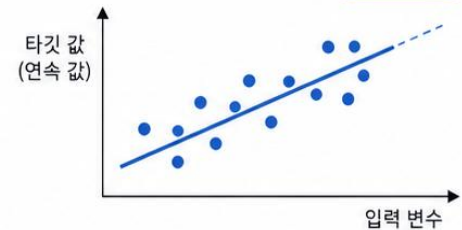
1 분류 (Classification)

입력 데이터를 사전에 정의된 이산적인 범주(Category) 중 하나로 할당하는 지도 학습 Task



2 회귀 (Regression)

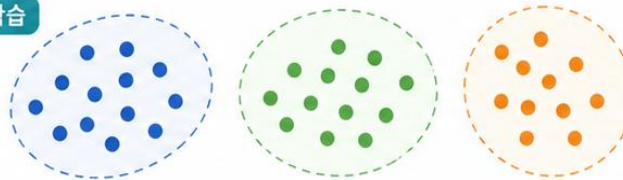
데이터 간의 상관관계를 분석하여 연속적인 수치(Continuous Value)를 예측하는 지도 학습 Task



3 군집화 (Clustering)

별도의 정답(Label) 없이 데이터 간의 유사성을 측정하여 유사한 특성을 가진 그룹으로 묶는 비지도 학습 Task

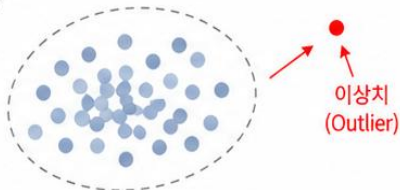
비지도 학습



4 이상 탐지 (Anomaly Detection)

대다수의 정상 데이터와 통계적으로 현저히 다른 패턴을 보이는 회귀 사례(Outlier)를 식별하는 Task

탐지 기반 Task



5 추천 (Recommendation)

사용자의 과거 행동 및 선호도 데이터를 기반으로 연관성이 높은 아이템의 우선순위를 결정하는 Task

개인화 기반 Task



분류(Classification) Task 예제 2가지

자율주행 시스템에서 객체와 신호를 올바르게 구분하는 대표 사례

1. 1. 전방 객체 분류 및 충돌 위험 식별



전방 보행자, 차량, 이륜차 등을 오인 없이
구분하여 충돌 위험을 식별

2. 2. 신호등 및 표지판 분류를 통한 법규 준수



신호등의 색상(적/황/녹) 및 표지판의 종류(정지, 제한 속도 등)를
분류하여 교통 법규를 준수



분류(Classification) Task

입력 데이터를 미리 정의된
여러 클래스 중 하나로 구분하는 작업



입력 데이터
(이미지, 영상 등)



모델(분류기)이
특징을 학습



정의된 클래스 중
하나로 분류

회귀(Regression) Task 예제 2가지

자율주행 시스템에서 연속적인 값을 정밀하게 예측하는 대표 사례

1. 선행 차량 거리 추정 및 부드러운 제동

연속값 예측



선행 차량과의 실제 거리를 정밀하게 추정하여 부드러운 제동 거리를 확보

2. 조향각 예측을 통한 차선 이탈 방지

연속값 예측



차선 이탈 방지를 위해 차량의 조향각(Steering Angle)을 도 단위로 세밀하게 계산하여 예측



회귀(Regression) Task

입력 데이터와 목표 값의 관계를 학습하여 연속적인 수치를 예측하는 작업



입력 데이터
(영상, 센서값 등)



모델이 패턴과 상관관계를 학습



연속적인 값
(거리, 각도 등) 예측



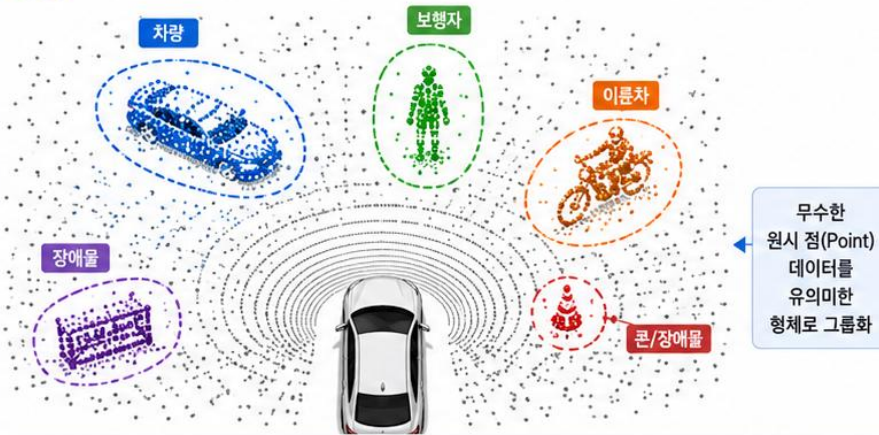
정밀 제어 및
안전 주행 지원

군집화(Clustering) Task 예제 2가지

자율주행 시스템에서 유사한 패턴과 객체를 그룹화하는 대표 사례

1. LiDAR 점군을 장애물 단위로 군집화

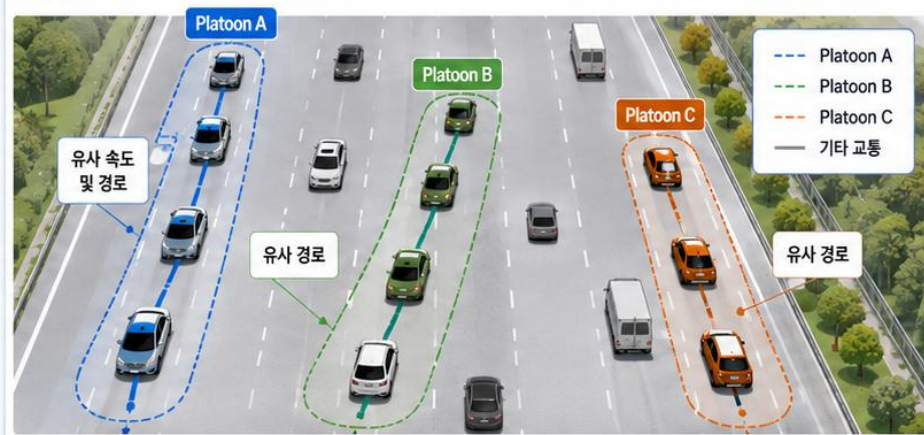
비지도 학습



LiDAR 센서의 무수한 점(Point) 데이터를 개별 장애물 단위로 묶어 형태를 식별

2. 주행 패턴 기반 차량군(Platoon) 그룹화

패턴 기반 군집화



주변 차량들의 주행 패턴을 분석하여 비슷한 경로를 가진 차량군(Platoon)으로 그룹화



군집화(Clustering) Task

정답(Label) 없이 데이터 간 유사성을 측정하여 비슷한 특성을 가진 그룹으로 묶는 작업



입력 데이터
(점군, 궤적 등)



유사성 기반
패턴 분석



비슷한 객체/차량군으로
군집화



주변 환경 인식 및
교통 흐름 파악

이상 탐지(Anomaly Detection) Task 예제 2가지

자율주행 시스템에서 비정상 객체와 거동 패턴을 감지하는 대표 사례

1. 미학습 도로 위험 요소 감지 및 비상 대응

OOD 탐지



도로 위의 낙하물, 싱크홀 등 미학습 객체(Out-of-Distribution)를 식별하여 비상 대응 체계를 가동

2. 비정상 주행 패턴 감지 및 위험 회피

행동 이상 탐지



주변 차량의 급작스러운 역주행이나 갈지자 주행 등 비정상적인 거동 패턴을 감지하여 위험을 회피



이상 탐지(Anomaly Detection) Task

정상 패턴과 현저히 다른 희귀 사례를 식별하여 위험 상황에 선제적으로 대응하는 작업



입력 데이터
(영상, 센서, 궤적 등)



정상 패턴 학습
및 기준 설정



이상 객체/행동
감지



경고, 비상 대응
및 위험 회피

추천(Recommendation) Task 예제 2가지

자율주행 시스템에서 개인화된 장소·경로 제안을 수행하는 대표 사례

1. 1. 연료량·취향 기반 최적 충전소/식당 추천

개인화 추천



주행 경로상의 잔여 연료량과 운전자 취향을 반영하여 최적의 충전소나 식당을 추천

2. 2. 교통 상황·운전 습관 기반 우회 경로 추천

실시간 경로 추천



현재 교통 상황과 운전자의 주행 습관을 고려하여 최적의 우회 경로를 실시간으로 추천



추천(Recommendation) Task

사용자 맥락과 선호를 분석하여 후보 대안의 우선순위를 정하고 최적의 선택지를 제안하는 작업



AI MODEL

Basic Terms

❖ Machine Learning Model

- mathematical construct that generates an inference or prediction based on input data or information

❖ Training, Model Training

- process to determine or to improve the parameters of a machine learning model, based on a machine learning algorithm, by using training data

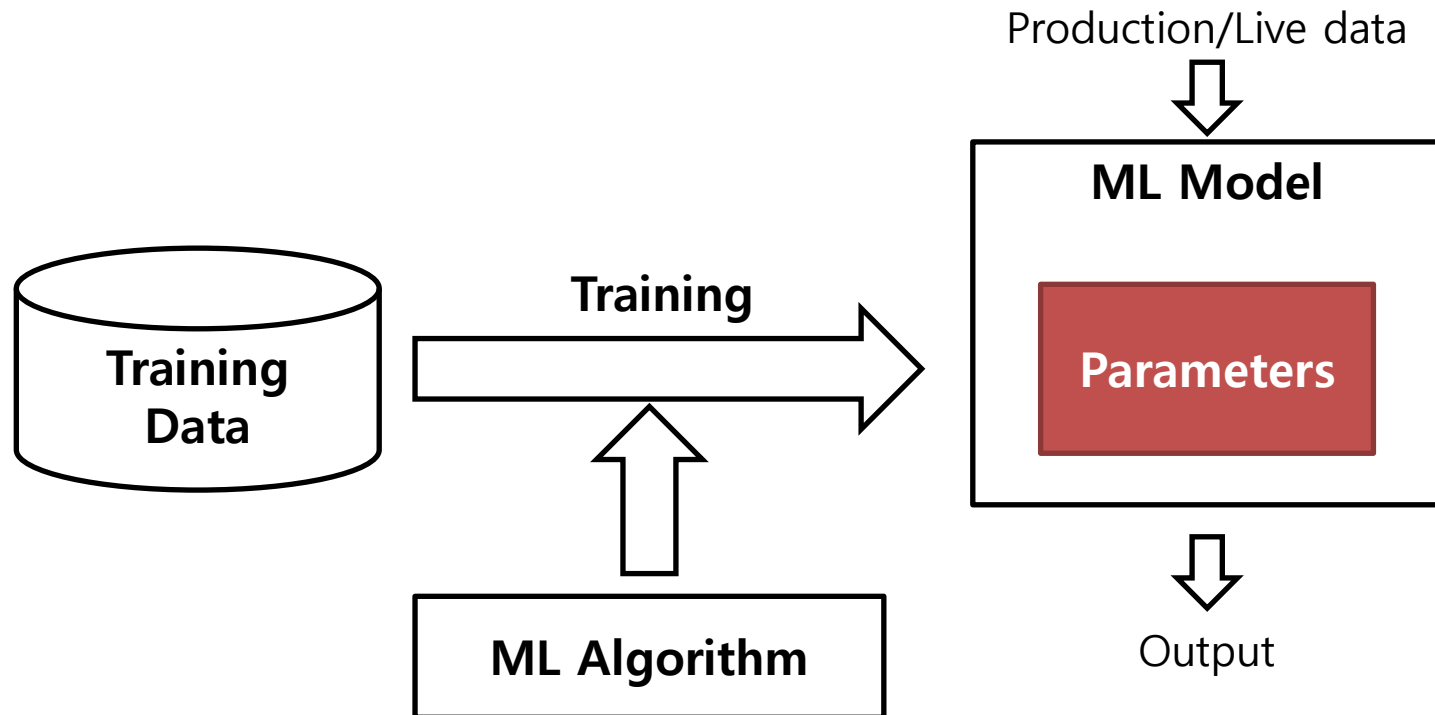
❖ Parameter, Model Parameter

- internal variable of a model that affects how it computes its outputs

❖ Machine Learning Algorithm

- algorithm to determine parameters of a machine learning model from data according to given criteria

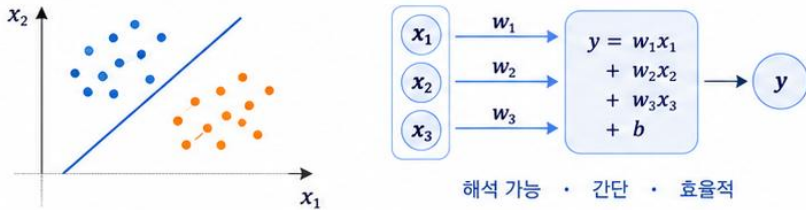
Basic Terms



ML Model 유형

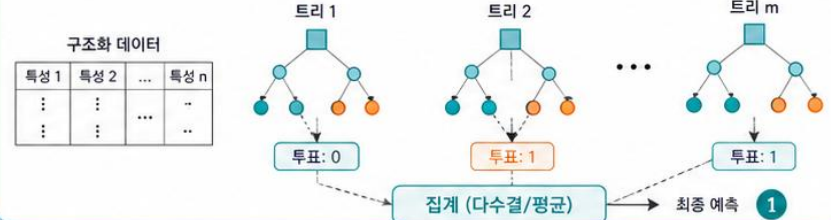
1 통계적 선형 모델 (Statistical Linear Models)

선형 결합과 확률적 가정으로 데이터를 해석하는 방식



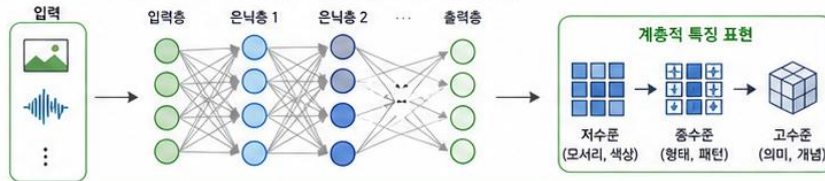
2 결정 트리 기반 앙상블 (Tree-based Ensembles)

여러 의사결정 나무를 결합해 비선형 패턴을 학습하는 방식



3 심층 신경망 (Deep Neural Networks)

다층 구조로 고차원 특징을 계층적으로 학습하는 방식



4 파운데이션 및 생성 모델 (Foundation & Generative)

대규모 사전학습으로 다양한 작업에 적응하고 새로운 콘텐츠를 생성하는 방식



5 거리 및 공간 유사도 모델 (Distance & Similarity)

벡터 공간에서 거리와 유사성으로 군집과 이웃을 찾는 방식



★ 핵심 관점: 선형성 · 트리 결합 · 계층적 표현학습 · 사전학습/생성 · 거리/유사도

Task 별 Model 선정 가이드

AI Task	주 적용 AI 모델	모델 선택 기준
 분류 (Classification)	XGBoost / LightGBM	→  정형 데이터 성능 극대화. CPU/GPU 병렬 처리가 필요하고 정확도가 최우선일 때.
	CNN / ViT	→  이미지, 영상 등 공간적 특징 추출이 필요한 비정형 데이터 처리 시.
	Logistic Regression	→  설명력이 필수적이고, 리소스가 제한된 환경일 때.
 회귀 (Regression)	Random Forest / CatBoost	→  데이터의 노이즈가 많고, 범주형 변수가 많은 정형 데이터 수치 예측 시.
	Ridge / Lasso	→  변수 선택을 자동화하고 모델 경량화가 필요할 때.
	LSTM / DeepAR / N-HiTS	→  시간적 순서가 중요하고, 외부 요인(Covariates)을 반영한 예측 시.
 군집화 (Clustering)	K-Means / Mini-Batch K-Means	→  데이터가 크고 군집 수가 명확할 때. 초기 베이스라인 구축용.
	DBSCAN / HDBSCAN	→  데이터 분포가 복잡하고 이상치에 민감한 군집화가 필요할 때.
	Hierarchical Clustering	→  데이터 간의 상관관계(계통도) 분석이 필요하거나 소규모 정밀 분석 시.
 이상 탐지 (Anomaly Detection)	Isolation Forest	→  대규모 데이터셋에서 고립된 이상치를 빠르고 가볍게 탐지할 때.
	AutoEncoder	→  정상 데이터의 패턴 재구성 오차를 이용해 미지의 이상 징후를 찾을 때.
	One-Class SVM	→  데이터가 고차원이고 정상/비정상 경계를 명확히 구고 싶을 때.
 추천 (Recommendation)	Matrix Factorization	→  사용자-아이템 간의 잠재 요인(Latent Factor) 분석 기반 고전적 추천.
	DeepFM / Wide & Deep	→  CTR(클릭률) 최적화와 도메인 지식(Wide)을 결합하고 싶을 때.
	Two-Tower (Retrieval)	→  밀집 벡터(Embedding) 검색을 통해 밀리초 단위로 후보를 뽑아야 할 때.

💡 데이터 특성(크기, 유형, 노이즈), 성능 목표(정확도, 속도), 운영 환경(리소스, 실시간 여부)을 종합적으로 고려하여 모델을 선택하세요.

Parameter vs Hyperparameter

구분	파라미터 (Parameter)	하이퍼파라미터 (Hyperparameter)									
용도 	학습 데이터를 바탕으로 결정된 지능을 수치화하여 표현하여 최종 추론 결과 도출에 사용  학습 데이터 →  학습 (최적화) → 파라미터 (가중치) $W, b, \dots$	용도 1) 학습 과정 제어: 학습의 속도, 수렴성 조절 용도 2) 모델의 아키텍처(구조) 결정: 모델의 용량, 층의 깊이 등  학습 과정 제어 (학습률, 배치크기 등)  모델 구조 결정 (층 수, 뉴런 수 등)									
의미 	모델의 지능 그 자체 추론 시 입력 데이터에 대한 계산에 사용되는 값 입력 데이터 (x) → 모델 (파라미터 사용)  → 예측 결과 ($\hat{y}$)	모델이 최적화 전략 및 설계도 모델 추론 목표 달성을 위한 학습 단계에서의 제어 변수 학습률 (Learning Rate) 배치 크기 (Batch Size) 에폭 수 (Epochs) 층 수 (Depth) 드롭아웃 비율 (Dropout) ...									
결정 방식 	모델 내부에서 학습 알고리즘을 통해서 결정(최적화) 초기값 → 순전파 → 손실 계산 → 역전파 → 파라미터 업데이트 (최적화) ↑	개발자의 지식(경험)으로 설정하거나 NAS, AutoML로 탐색  개발자 경험 (수동 설정) 또는  탐색 (검색) —또는—  NAS / AutoML (자동 탐색)									
배포 	직렬화된 모델 아키텍처 및 가중치 데이터로 배포 모델 아키텍처 (구조)  + 파라미터 (가중치) $W, b, \dots$ <table><tr><td>0.12</td><td>-0.35</td><td>...</td></tr><tr><td>0.07</td><td>0.89</td><td>...</td></tr><tr><td>...</td><td>...</td><td>...</td></tr></table> → 배포 (추론 서버/디바이스) 	0.12	-0.35	...	0.07	0.89	...	...	...	...	모델의 아키텍처 모델 구조로 고정되어, 설정 파일로 배포  JSON  설정 파일 학습 과정 제어 학습 완료 후 폐기 (추론 시 불필요) 
0.12	-0.35	...									
0.07	0.89	...									
...	...	...									
전체 흐름 요약 	1. 하이퍼파라미터 설정 (설계도 및 학습 전략 결정) → 2. 모델 학습 (파라미터 최적화) → 3. 파라미터 확정 (지능이 수치로 결정) → 4. 모델 배포 (추론에 사용) 하이퍼파라미터 튜닝 (반복) 학습 반복 (최적화)		핵심 요약 - 파라미터 = 모델의 지능 (학습으로 결정, 추론에 사용) - 하이퍼파라미터 = 모델의 설계도 및 학습 전략 (개발자가 결정) - 좋은 하이퍼파라미터 → 좋은 파라미터 학습 → 좋은 성능								

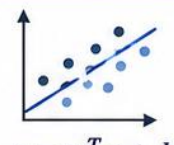
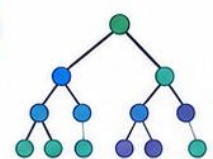
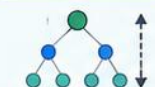
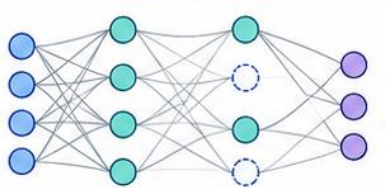
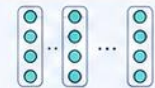
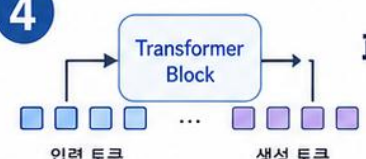
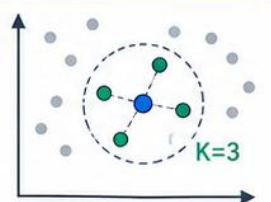
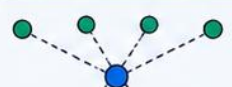
심층 신경망

구분	요소	역할	비고 (구조 vs 제어)
파라미터	가중치 (Weight)	각 입력 신호가 결과에 미치는 영향력(강도)	학습 결과물 (배포 필수)
	편향 (Bias)	뉴런의 활성화 조건을 조절하는 임계값	학습 결과물 (배포 필수)
하이퍼-파라미터	은닉층 수 / 노드 수	신경망의 깊이와 너비 (모델의 용량 결정)	모델 구조 (설정 파일 배포)
	학습률 (Learning Rate)	가중치를 한 번에 수정하는 보폭 (수렴 속도 조절)	학습 제어 (학습 후 폐기)
	배치 크기 (Batch Size)	가중치 1회 업데이트에 사용할 샘플 개수	학습 제어 (학습 후 폐기)
	드롭아웃 (Dropout)	과적합 방지를 위해 무작위로 비활성화할 뉴런 비율	학습 제어 (추론 시 끄/폐기)

랜덤 포레스트 (Random Forest)

구분	요소	역할	비고 (구조 vs 제어)
파라미터	분할 변수 및 임계점	노드에서 데이터를 나누는 최적의 기준 (예: Age > 30)	학습 결과물 (배포 필수 / 트리 노드에 내장)
	리프 노드의 값	나무의 끝단에서 결정된 최종 예측값 (클래스 또는 평균값)	학습 결과물 (배포 필수 / 최종 출력 데이터)
하이퍼-파라미터	학습기 개수 (n_estimators)	앙상블에 참여시킬 의사결정 나무의 총 개수	모델 구조 (설정 파일 배포 / 전체 모델 규모 결정)
	최대 깊이 (Max Depth)	각 나무가 수직으로 성장할 수 있는 최대 층수 제한	모델 구조 (설정 파일 배포 / 추론 경로 복잡도 결정)
	최소 분할 샘플 수	노드 분기를 위해 필요한 최소 데이터 개수 (가지치기)	학습 제어 (학습 후 폐기 / 트리 형성 공정에만 관여)
	최대 피쳐 수 (max_features)	각 노드 분기 시 후보로 고려할 특성(Feature)의 개수	학습 제어 (학습 후 폐기 / 나무 간 다양성 확보용)
	부트스트랩 샘플링	학습 데이터 구축 시 무작위 중복 추출 사용 여부	학습 제어 (학습 후 폐기 / 데이터 샘플링 전략)

모델 별 하이퍼 파라미터

1  통계적 선형 모델 $y = w^T x + b$	규제 강도 $\lambda \rightarrow$  계수 축소	모델 가중치에 페널티를 주어 과적합을 방지함 (L1/L2 Regularization).
2  결정 트리 기반 앙상블	1 최대 깊이 / 학습기 수 2 최소 분할 샘플 수	 개별 트리의 수직적 복잡도 및 결합할 기초 모델(Tree)의 총 개수를 결정함.  노드를 나누기 위한 최소 데이터 수. 불필요한 가지 생성을 억제(가지치기)함.
3  심층 신경망	1 레이어 수 / 노드 수 2 드롭아웃 비율	 신경망의 깊이와 너비를 결정하여 모델의 학습 용량을 조절함.  학습 시 무작위로 뉴런을 비활성화하여 특정 뉴런에 대한 의존도를 낮추고 견고성 향상.
4  파운데이션 및 생성 모델 입력 토큰 생성 토큰	1 컨텍스트 길이 / 샘플링 스텝 2 온도 (Temperature)	 모델이 참조하는 최대 토큰 범위 또는 생성 시 노이즈 복원을 위한 반복 횟수 설정. 낮음 온도 높음 일관성 ↑ 창의성 ↑ 출력 확률 분포를 조절하여 결과물의 창의성(무작위성)과 일관성 사이의 균형을 맞춤.
5  거리 및 공간 유사도 모델 K=3	1 이웃 수 (K) / 잠재 요인 차원 2 거리 측정 공식	 유사도 판단을 위한 근접 포인트 개수 또는 데이터 특징을 압축할 벡터의 차원 수 설정. $d_{euclid}(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$ 유클리드, 코사인 등 공간상에서 $d_{cosine}(x, y) = 1 - \frac{x \cdot y}{\ x\ \ y\ }$ '유사성'을 정의하는 수리적 방식을 선택함.

MODEL/SOFTWARE 배포

AI 모델 형식(Format)



서버 /
범용

1 Safetensors

주요 타겟: LLM, Diffusion

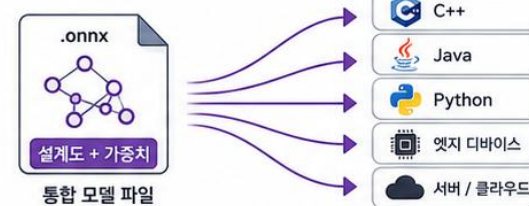


- ✓ 가중치 전용
- ✓ 보안
- ✓ 빠른 로딩
- ✓ 서버 배포 표준

가중치만 안전하게 담아 서버 환경에서 배포할 때 적합

2 ONNX

주요 타겟: 고전 ML, CNN, RNN



- ✓ 플랫폼 독립성
- ✓ 설계도 + 가중치
- ✓ 다양한 런타임
- ✓ 범용 교환 포맷

고전 ML부터 딥러닝까지 다양한 환경에서 구동 가능한 통합 포맷



엣지 /
임베디드

3 GGUF

주요 타겟: LLM 중심



- ✓ 로컬 실행 최적화
- ✓ CPU/GPU 효율
- ✓ 경량화
- ✓ LLM 추론

개인용 PC나 엣지 서버에서 대형 언어모델을 효율적으로 실행

4 TFLite

주요 타겟: 고전 ML, CNN



- ✓ 모바일/임베디드
- ✓ 경량 모델
- ✓ 저전력
- ✓ 현장 추론

안드로이드와 IoT 기기에서 가벼운 모델 실행에 적합



핵심 관점



Safetensors

보안 · 가중치 전용



ONNX

호환성 · 범용성



GGUF

로컬 최적화 · LLM



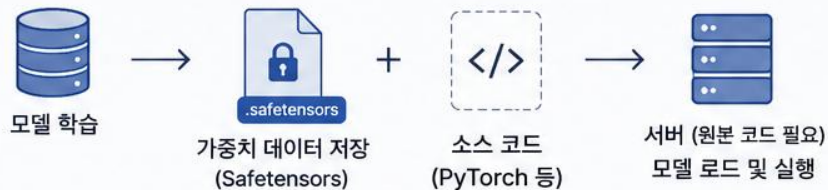
TFLite

모바일 · 임베디드

서버 배포: Safetensors (데이터 중심) vs ONNX (연산/실행 중심)

구분	Safetensors (데이터 중심)	ONNX (연산/실행 중심)
 핵심 개념	모델의 가중치(Weights) 데이터만 안전하게 저장	가중치 + 연산 그래프(Graph)를 표준화하여 통합 저장
 비유 (Software)	가중치 데이터 파일 (소스 코드 별도 필요)  +  소스 코드 (PyTorch 등)	컴파일된 바이너리 (JRE와 같은 런타임에서 즉시 실행)  →  런타임 엔진 (ONNX Runtime)
 포함 요소	순수 텐서(Tensor) 데이터 	계산 노드(연산 로직) + 가중치 데이터 
 실행 독립성	낮음: 원본 소스 코드(PyTorch 등)와 결합 필수	높음: 별도 코드 없이 런타임 엔진만으로 구동 가능
 범용성/호환성	PyTorch/Hugging Face 생태계 중심	매우 높음: C++, C#, Java, Rust 등 다양한 환경 지원
 로딩 속도	매우 빠름: Zero-copy 기술로 메모리 즉시 로드	보통: 그래프 해석 및 하드웨어 최적화 단계 필요
 보안성	매우 우수: 실행 코드가 없어 악성 코드 삽입 불가	우수: 표준 프로토콜 기반이나 파일 구조가 상대적으로 복잡함

Safetensors 배포 흐름 (데이터 중심)



가중치 파일 + 소스 코드가 결합되어야 실행 가능

ONNX 배포 흐름 (연산/실행 중심)



단일 파일만으로 런타임에서 즉시 실행 가능

로컬 에지 배포: GGUF vs TFLite

구분	GGUF	TFLite
 개발 주체	Georgi Gerganov (커뮤니티 중심)	 Google
 타겟 플랫폼	 로컬 PC (Mac/Windows/Linux), 엣지	 Android, 모바일, 임베디드, 브라우저
 핵심 특징	 단일 파일 배포: 가중치와 메타데이터를 하나로 통합. CPU 기반 고속 양자화 추론 특화.	 모바일 범용성: 안드로이드 생태계의 표준. 하드웨어 가속(GPU, NNAPI) 지원.
 주요 장점	 로컬 환경에서 대규모 모델(LLM)을 적은 메모리로 구동 가능.	 모델 크기가 매우 작고 모바일 기기 호환성이 뛰어남.
 호환 프레임워크	llama.cpp 기반 엔진	TensorFlow, PyTorch(변환 필요)
 보안성	우수 (순수 데이터 및 메타데이터)	우수 (바이너리 포맷)
 주요 용도	 로컬 LLM 구동 (Llama 3, Mistral 등)	 안드로이드 앱 내 실시간 비전/음성 인식

GGUF 배포 흐름 (로컬/엣지 중심)



TFLite 배포 흐름 (모바일/임베디드 중심)



요약

GGUF는 로컬에서 대규모 LLM을 효율적으로 실행하기 위한 CPU 최적화 포맷,
TFLite는 모바일/임베디드 환경에서 실시간 AI 서비스를 제공하기 위한 경량 최적화 포맷입니다.

Parameter 및 Hyperparameter(모델 아키텍처) 배포 방식

배포 형식 (Format)

파라미터 (Parameter / 가중치)

하이퍼파라미터 (Hyperparameter / 구조 등)

비고 (구동 방식의 차이)

Safetensors

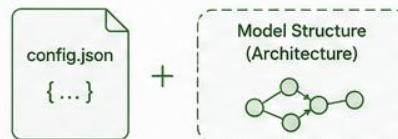


.safetensors

순수 텐서 데이터만 포함.
보안과 로딩 속도 최적화를 위해
바이너리 형태로 저장.



포함되지 않음.
레이어 수, 헤드 수 등 구조 정보는
외부 파일(config.json)로 분리 관리함.



[데이터-구조 분리형]

실행 시 가중치 파일과
구조 정의 파일(JSON)을
결합하는 로직 필수.



ONNX

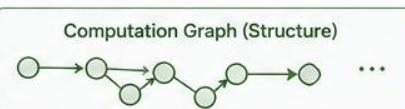


.onnx

바이너리 데이터로 포함.
Initializers 영역에 가중치 행렬이
직렬화되어 저장됨.



연산 그래프 형식으로 포함.
노드 간 연결 순서와 활성화 함수 등
구조 정보가 파일 내부에 정의됨.



[데이터-구조 통합형]

소스 코드 없이도 ONNX Runtime을
통해 즉시 모델 복원 및 실행 가능.



GGUF



.gguf

양자화된 텐서 데이터 포함.
4-bit, 8-bit 등 압축된 가중치를
단일 파일에 저장.



키-값(KV) 쌍으로 포함.
모델명, 구조 정보, 양자화 설정 등이
단일 파일의 헤더 영역에 기록됨.



[올인원(All-in-One)]

LLM 구동에 필요한 모든 정보가
단일 파일에 있어 배포 및 공유가 매우 간편함.



TFLite



.tflite

FlatBuffers 형식으로 포함.
모바일 환경에 최적화된 압축
가중치 데이터 저장.



연산 그래프 형식으로 포함.
모바일 기기의 NPU/GPU 가속을
고려한 최적화된 구조 정보 포함.



[엣지 최적화 통합형]

엣지 기기의 런타임(Interpreter)이
파일 내부의 구조와 데이터를 읽어
즉시 추론 수행.



Model Artifact vs Package Model(or Model Package)

	Model Artifact (지능의 본체)	VS	Packaged Model (지능의 운용체)
 핵심 가치	 학습 결과물로서의 지능 보존 및 추적		 운영 환경에서의 독립적 실행 및 서빙 보장
 구성 요소	 Parameters (가중치)  Architecture (구조)  Metadata (학습 이력, 성능 지표 등)		 Artifact (Inclusion/Reference) +  Inference Code Data Prep Logic +  Guardrails (안전/정책/검증) +  Serving API (REST/gRPC)
 추상화 수준	 Static Asset (정적인 데이터 자산)		 Active Asset (동적으로 동작하는 소프트웨어 자산)
 배포 단위	 Model Registry에 버전별로 관리 (예: MLflow, DVC) 		 Container Registry에 이미지로 관리 (예: Docker, Kubernetes) 
 엔지니어링 특성	 데이터 무결성 및 버전 관리(Lineage)가 중요 		 의존성 관리(Dependency) 및 확장성(Scalability)이 중요  Dependencies 관리  Auto Scaling 및 확장성
	Model Artifact = 지능 그 자체 (What) 모델이 '무엇을' 알고 있는지에 대한 기록이자 증거 (진실의 원본)		 Packaged Model = 지능을 '실행하는 방법' (How) 모델이 '어떻게' 환경에서 안전하고 안정적으로 동작할지에 대한 패키지

Q & A
